

DAY 3 · LECTURE 2 · 90 MINUTES

The Captain's Log

GitOps with Gitea & ArgoCD

This morning Helm wrote your blueprints. This afternoon, we take your hands off the wheel.

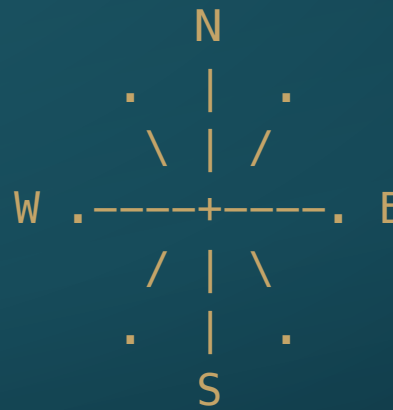


Where we are

- ♦ This morning: **Helm** let you mass-produce manifests — one chart, thirty namespaces.
- ♦ But you were still running `helm install` by hand from your own terminal.
- ♦ This afternoon we remove the laptop from the loop — **entirely**.
- ♦ Four parts:
 - **Part I** — Git Is Truth (the mental model)
 - **Part II** — The CI Loop (Gitea & Harbor)
 - **Part III** — The CD Engine (ArgoCD)
 - **Part IV** — Wiring the Archipelago (Network as Code)
- ♦ Then: **Lab 02 — The Automated Shipyard** and **Lab 03 — Wiring the Archipelago**.

PART I

Git Is Truth



"All I ask is a tall ship and a star to steer her by." — John Masefield, "Sea-Fever"

The problem with hand-steering

- ♦ This morning's workflow: write a chart → run `helm install` → done.
- ♦ That is still a human at a terminal, making a one-time command.
- ♦ Problems that don't go away:
 - Who ran the last deploy? From which machine? With which values?
 - A student edits a Deployment by hand — now the cluster disagrees with the chart.
 - You push a fix — but did it actually land everywhere?
- ♦ **Every change is tribal knowledge.** Nothing is written down authoritatively.

The Captain's Log

```
+-----+
| Captain's Log (Git repo) |
| "What SHOULD be true"   |
| desired state -- in writing |
+-----+
|
|   GitOps controller reads it
|   v
+-----+
| Crew's Actions (cluster) |
| "What IS true right now"  |
| live state -- always moving |
+-----+
```

- ♦ The **Git repo is the Captain's Log** — the authoritative, written record of intent.
- ♦ The **live cluster is the Crew's Actions** — what is actually running.

The GitOps commandment

Git is Truth.

- ♦ The Log is authoritative. If the crew's actions differ from the log — the **log wins**.
- ♦ A **GitOps controller** sits between them. Its only job:
 - continuously **read the Log**
 - compare what the cluster **is** to what the log **says it should be**
 - make the cluster **match** — automatically, without you
- ♦ Every change to the system is a **Git commit** — reviewable, auditable, reversible.

What changes — and what doesn't

Before GitOps	After GitOps
<code>helm install</code> from your laptop	<code>git push</code> — the controller deploys
"Did it deploy?" — check Slack	ArgoCD UI — green or red, always
A student hand-edits a resource	ArgoCD flags it OutOfSync , self-heals
Rollback means re-running commands	<code>git revert</code> — one commit undoes it

The cluster becomes a read-only reflection of Git.

No more `kubectl apply` from terminals

- ♦ Manual `kubectl apply` is like a sailor changing course without logging it.
- ♦ GitOps says: **if it is not in Git, it does not exist.**
 - Want a new Deployment? Commit a manifest.
 - Want to change a replica count? Edit `values.yaml` and push.
 - Want to roll back? `git revert` — the controller does the rest.
- ♦ The laptop is no longer the source of truth. **The repo is.**

PART II

The CI Loop



"You don't haul cargo by hand every voyage. Build the crane once and let it load itself." — the Boatswain

The problem: still pushing by hand

- ♦ Lab 01 (Day 1) workflow: write code → `docker build` → `docker push` → done.
- ♦ That push happened from your laptop — manually, every single time.
- ♦ Problems:
 - Did you forget to push the latest version?
 - Did the image tag match the chart?
 - Which student's build is actually in Harbor right now?
- ♦ **Continuous Integration (CI)** automates the build-and-push step so you never ask those questions.

Why a self-hosted Git server?

- ♦ The island's Git server is **Gitea**, running at `gitea.wagbiz.org`.
- ♦ Why not GitHub or GitLab cloud?
 - The lab runs in a **closed, offline environment** — no public internet.
 - Images and source code stay **inside the cluster**, never leaving.
 - The registry (Harbor) and the Git server speak the same internal network.
- ♦ Self-hosted Git is also the realistic picture for many institutional IT environments.

The island's toolbox

```
gitea.wagbiz.org  -- the island's Git server
harbor.wagbiz.org -- the island's container registry
argocd.wagbiz.org -- the GitOps controller (Part III)
```

- ♦ **Gitea** — push and pull code; pipelines live here (Gitea Actions).
- ♦ **Harbor** — store and serve container images to the cluster.
- ♦ **ArgoCD** — watch Gitea repos and sync the cluster to match.
- ♦ Three services. One closed loop. No laptop required.

The CI pipeline: code in, image out

```
Developer
|
| git push
v
gitea.wagbiz.org
|
| Gitea Actions trigger
v
CI Runner (inside the cluster)
| docker build
| docker push
v
harbor.wagbiz.org
|
| image available to cluster
v
(ArgoCD picks it up -- Part III)
```

Gitea Actions — the pipeline file

- ♦ Gitea Actions is **GitHub-Actions-compatible** — same YAML syntax, familiar to many.
- ♦ A pipeline lives at `.gitea/workflows/build.yaml` in your repo.
- ♦ The essentials:
 - `on: push` — trigger on every push to `maindeck`
 - `docker build` — produce the image
 - `docker push` — send it to `harbor.wagbiz.org`
- ♦ Push once. Harbor has the fresh image. ArgoCD can deploy it.

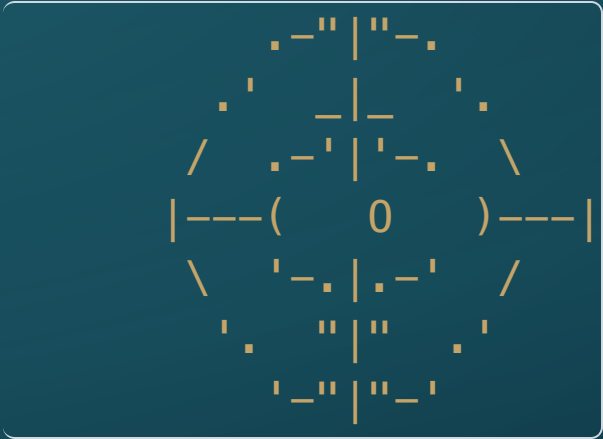
The branch: `maindeck`

- ♦ On this island the trunk branch is called `maindeck` — not `main`.
- ♦ Trunk-based development: the whole crew commits to one authoritative deck.
- ♦ In class: no long-lived feature branches. Commit, push, watch it deploy.
- ♦ The `maindeck` name is nautical flavor — the concept applies to any trunk branch strategy.

The log has one authoritative page. Everyone writes to the same deck.

PART III

The CD Engine



```
      .- " | " -.  
      '  _ | _  '  
    /  .- ' | ' - \  \  
 |---(    0    )---|  
    \  ' - . | . - ' /  \  
      '  " | "  '  '  
      ' - " | " - '
```

"The helmsman doesn't keep asking which way to steer. He reads the chart once and holds it."
— the Boatswain

Continuous Delivery: what it means

- ♦ **CI** built and pushed the image — automatically.
- ♦ **CD** deploys it — automatically.
- ♦ The full loop:
 - i. Commit a chart or config change to Gitea
 - ii. ArgoCD **notices** the change
 - iii. ArgoCD **syncs** the cluster to match Git
 - iv. Live state now equals desired state — green
- ♦ No one ran `helm upgrade`. No one ran `kubectl apply`. **Git was the trigger.**

ArgoCD: one job, done obsessively

- ♦ ArgoCD watches a Git repo on a **polling interval** (configurable; typically ~3 minutes) or via a **webhook** for instant triggers.
- ♦ On every check, it computes the diff: desired state (Git) vs. live state (cluster).
- ♦ **Synced** — the cluster matches Git. All green.
- ♦ **OutOfSync** — something disagrees. ArgoCD flags it.
- ♦ With **Self Heal** enabled: ArgoCD corrects the drift automatically, without waiting for you.

The ArgoCD Application object

```
apiVersion: argoproj.io/v1alpha1
kind: Application
spec:
  source:
    repoURL: http://gitea-http.admin-tools.svc...
    targetRevision: maindeck
    path: .
  destination:
    namespace: <your-name>
  syncPolicy:
    automated:
      selfHeal: true
      prune: true
```

- ♦ This object is itself just a Kubernetes manifest — GitOps all the way down.
- ♦ `selfHeal: true` — drift is corrected automatically.
- ♦ `prune: true` — objects removed from Git are removed from the cluster.

The reconciliation loop

```
Git repo (desired state)
|
|  ArgoCD reads
v
Compare: desired vs. live
|
+-- MATCH --> Synced (green)
|
+-- DRIFT --> OutOfSync
      |
      +-- Self Heal ON --> auto-sync back
      |
      +-- Self Heal OFF --> wait for manual Sync
```

ArgoCD runs this loop continuously. It never stops watching.

Drift detection: The Mutiny

- ♦ A student hand-edits a Deployment and scales replicas to 10 — behind the shipyard's back.
- ♦ ArgoCD sees: Git says 1 replica. Cluster says 10. **OutOfSync**.
- ♦ Self Heal fires. Replicas reset to 1. The mutiny is over.
- ♦ Now try: `kubectl delete deployment` on an ArgoCD-managed app.
- ♦ The pod respawns — because **Git still says it should exist**.

The only way to change the fleet is to change the Log.

Rollback is just `git revert`

- ◆ Something broke in the latest push?
- ◆ You do not scramble to remember the previous Helm values.
- ◆ You `git revert` the commit — and push.
- ◆ ArgoCD reads the reverted log. Syncs the cluster back to the previous state.
- ◆ **Rollback becomes a version-control operation**, not an operational fire drill.

The log is the record. The cluster is the consequence.

What the ArgoCD UI tells you

- ♦ **Application cards** — one per ArgoCD Application; shows Healthy/Degraded and Synced/OutOfSync at a glance.
- ♦ **App graph** — every Kubernetes object owned by the app, live status.
- ♦ **History tab** — every sync, linked to the Git commit that triggered it.
- ♦ **Sync button** — force an immediate reconciliation (or wait for Self Heal).
- ♦ `argocd.wagbiz.org` — log in with the credentials your instructor provided.

The full GitOps loop — assembled

```
[you edit values.yaml]
|
| git push to gitea.wagbiz.org
v
[Gitea Actions triggers]
|
| docker build + push to harbor.wagbiz.org
v
[ArgoCD polls / receives webhook]
|
| diff: desired vs. live
v
[ArgoCD syncs cluster]
|
v
[students see updated app -- no manual step]
```


PART IV

Wiring the Archipelago



"No man is an island, entire of itself." — John Donne, Meditation XVII

Beyond apps: Network as Code

- ♦ So far: GitOps deploys **web apps** — Deployments, Services, Ingresses.
- ♦ But Kubernetes can deploy far more than that, via **Custom Resource Definitions (CRDs)**.
- ♦ A CRD teaches Kubernetes a new kind of object — one the cluster didn't know about before.
- ♦ With the right CRD installed, you can deploy **commercial router nodes** the same way you deploy a Pod.
- ♦ **Network as Code** works exactly like **App as Code**.

Clabernetes: real routers inside Kubernetes

- ♦ **Clabernetes** runs **Containerlab** topologies *inside* Kubernetes.
- ♦ You write a **Topology** manifest — Kubernetes YAML — describing the router nodes and the links between them.
- ♦ Clabernetes turns each node into a running Pod with a **real commercial CLI** inside.
- ♦ The **links:** field in the manifest *is* the physical cable between routers.
- ♦ Supported nodes: **Nokia SR Linux**, **Cisco**, and more.

The Topology manifest

```
apiVersion: clabernetes.containerlab.dev/v1alpha1
kind: Topology
metadata:
  name: archipelago
spec:
  definition:
    containerlab: |
      topology:
        nodes:
          router-port:
            kind: nokia_srlinux
          router-starboard:
            kind: nokia_srlinux
        links:
          - endpoints:
              ["router-port:e1-1", "router-starboard:e1-1"]
```

- ♦ Commit this to Gitea. ArgoCD deploys it. Two router Pods appear — connected.

Deployed the same way as everything else

- ♦ Add `archipelago.yaml` to your Gitea repo.
- ♦ ArgoCD syncs — Clabernetes creates the router Pods.
- ♦ Check `k9s`: `archipelago-router-port` and `archipelago-router-starboard` come up.
- ♦ `kubectl exec` into a router pod → you are at a **Nokia SR Linux commercial CLI**.
- ♦ Configure interfaces, loopbacks, static routes — and `ping` across the link.

Same GitOps loop. Same commit. Same ArgoCD. Just a very different kind of workload.

What Lab 03 looks like in practice

- ♦ Depending on pacing, Lab 03 runs either as a **student lab** or an **instructor demo**.
- ♦ Either way the sequence is the same:
 - i. Commit the `Topology` manifest to Gitea
 - ii. ArgoCD syncs → router Pods appear
 - iii. `kubectl exec` into each router → configure IP addresses
 - iv. Add a static route → `ping` the other router's loopback
- ♦ When the ping replies: two commercial routers, talking across a route you charted, all running inside Kubernetes, all deployed by GitOps.

Instructor Superpower

THE LIVE CLASSROOM DEMO

- ♦ You have an ArgoCD app watching a JupyterLab chart in Gitea.
- ♦ Edit the chart's `requirements.txt` — add one line: `pandas`.
- ♦ Commit and push to Gitea.
- ♦ ArgoCD re-syncs every student's Jupyter environment.
- ♦ **No student typed `pip install`**. You distributed a heavy data-science library to the entire class by editing one file.

You just shipped a lab with a Git commit.

Instructor Superpower

REPLACING THE HARDWARE LAB

- ♦ Teaching BGP, OSPF, or EVPN has historically required:
 - GNS3 or EVE-NG (heavy, slow, hard to manage at scale)
 - Expensive proprietary physical hardware
 - An IT ticket to get any of it provisioned
- ♦ With **Clabernetes + ArgoCD**: commit a YAML file → every student has a full multi-router lab.
- ♦ No GNS3. No EVE-NG. No physical hardware.
- ♦ A full router lab used to mean a hardware budget and a provisioning request. Now it is a YAML file you commit yourself.

Both superpowers, one lens

- ♦ Step back from the two demos — the shift underneath them is the same.
- ♦ The environments you can choose to support have radically expanded: a data-science stack, a multi-router lab, a whole course. You don't need anyone's permission to build one.
- ♦ Define it in Git and it isn't just yours — IT can read it, reproduce it, and version it like any other codebase. The repo becomes a language both sides share.

The Captain's Log isn't just for the cluster. It's for the institution.

Up next: Lab 02 & Lab 03

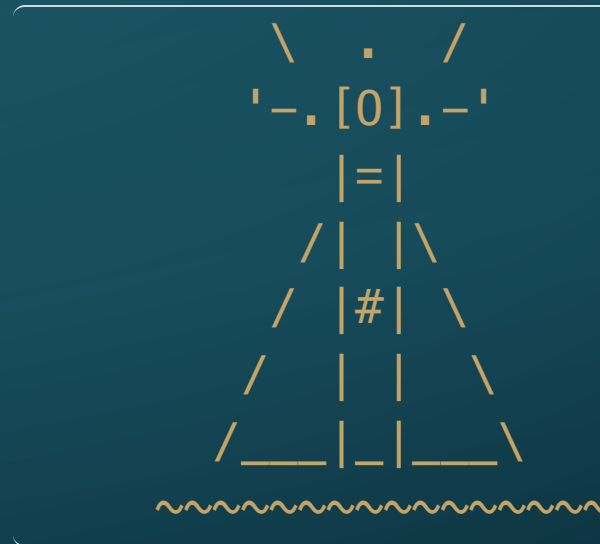
Lab 02 — The Automated Shipyard (hands-on, everyone)

- ♦ Push your Helm chart from Lab 01 to a Gitea repo.
- ♦ Create an ArgoCD Application watching that repo.
- ♦ Trigger The Mutiny — manually scale a Deployment, watch ArgoCD self-heal.
- ♦ Observe the Live Classroom Demo from the podium.

Lab 03 — Wiring the Archipelago (hands-on or instructor demo, depending on pace)

- ♦ Commit a Clabernetes `Topology` manifest to Gitea.
- ♦ ArgoCD deploys two Nokia SR Linux router Pods.
- ♦ Configure interfaces, chart a static route, ping across the link.

Write it down. Push it. Trust the Log.



The laptop leaves the loop. The Log takes the wheel.

